

Practical No-1

Q.1) Create a simple project using any programming language and perform operations using Git

Step 1: Create project folder

```
mkdir myproject
```

```
cd myproject
```

Step 2: Create a simple Python file

```
echo "print('Hello DevOps!')" > app.py
```

Step 3: Initialize Git repository

```
git init
```

Step 4: Add and commit

```
git add app.py
```

```
git commit -m "Initial commit: Added simple Python file"
```

Step 5: Check repository info

```
git status
```

```
git log --oneline
```

Q.2) Create a simple Java project using Maven, add dependencies, configure POM, compile code

Step 1: Install Maven if not installed

```
sudo apt update
```

```
sudo apt install maven -y
```

Step 2: Create Maven project

```
mvn archetype:generate -DgroupId=com.example -DartifactId=maven-demo \
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```
cd maven-demo
```

Step 3: Edit POM file (optional using nano)

```
nano pom.xml
```

Add JUnit dependency if needed

Step 4: Compile and test

```
mvn compile
```

```
mvn test
```

Step 5: Package to create JAR

```
mvn package
```

```
ls target/
```

Practical NO -2

Q.1) Docker and Containerization

Task 1: Dockerfile Creation and Build

Task 2: Running and Managing Containers

Step 1: Install Docker (if not installed)

```
sudo apt install docker.io -y
```

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

Step 2: Create project folder and HTML

```
mkdir docker-html
```

```
cd docker-html
```

```
echo "<h1>Hello DevOps from Nginx Docker</h1>" > index.html
```

Step 3: Create Dockerfile

```
cat <<EOF > Dockerfile
```

```
FROM nginx:latest
```

```
COPY index.html /usr/share/nginx/html/index.html
```

```
EXPOSE 80
```

```
EOF
```

Step 4: Build image

```
sudo docker build -t mynginxapp .
```

Step 5: Run container

```
sudo docker run -d -p 8080:80 --name mynginxcontainer mynginxapp
```

Step 6: Verify

```
sudo docker ps
```

```
curl http://localhost:8080
```

Step 7: Stop and remove container

```
sudo docker stop mynginxcontainer
```

```
sudo docker rm mynginxcontainer
```

Q.2) Create a simple project and push it to GitHub using Git

Step 1: Create folder

```
mkdir gitproject
```

```
cd gitproject
```

```
echo "print('Welcome to DevOps Project')" > app.py
```

Step 2: Initialize Git and commit

```
git init
```

```
git add app.py
```

```
git commit -m "Initial commit Added Python file"
```

Step 3: Connect to GitHub repo

```
git remote add origin https://github.com/<your-username>/devops-project.git
```

Step 4: Push to GitHub

```
git push -u origin master
```

Step 5: Display commit history

```
git log --oneline
```

Practical No-3

Q.1) Applying CI/CD Principles to Web Development Using Jenkins, Git, and Local HTTP Server (Apache or Nginx)

Step 1: Install Jenkins and Nginx

```
sudo apt update
```

```
sudo apt install openjdk-17-jdk nginx -y
```

```
wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo apt-key add -
```

```
sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ > \
```

```
/etc/apt/sources.list.d/jenkins.list'
```

```
sudo apt update
```

```
sudo apt install jenkins -y
```

```
sudo systemctl enable jenkins
```

```
sudo systemctl start jenkins
```

Step 2: Create simple HTML project

```
mkdir webproject
```

```
cd webproject
```

```
echo "<h1>Hello DevOps CI/CD with Jenkins</h1>" > index.html
```

```
git init
```

```
git add index.html
```

```
git commit -m "Initial commit for Jenkins CI/CD"
```

```
git remote add origin https://github.com/<your-username>/webproject.git
```

```
git push -u origin master
```

```
# Step 3: Open Jenkins in browser

# URL: http://localhost:8080

# Configure new Freestyle job linked to your GitHub repo

# In build step, use command:

# cp -r * /var/www/html/
```

Q.2) Create a simple project, push it to GitHub, create a new branch, merge it, and show commit history

```
# Step 1: Create project
mkdir branchproject
cd branchproject
echo "print('Branch project start')" > app.py
git init
git add app.py
git commit -m "Initial commit Added app.py"

# Step 2: Connect and push
git remote add origin https://github.com/<your-username>/branchproject.git
git push -u origin master

# Step 3: Create branch and modify
git checkout -b featurebranch
echo "print('Feature added')" >> app.py
git add app.py
git commit -m "Added featurebranch update"

# Step 4: Merge and push
git checkout master
git merge featurebranch
git push origin master

# Step 5: Show history
git log --oneline
```

Practical No-4

Q.1) Configure Maven to compile the code, run tests, and generate artifacts like JAR files

```
# Step 1: Create project
mvn archetype:generate -DgroupId=com.example -DartifactId=mavenproject \
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

cd mavenproject
# Step 2: Compile
mvn compile
# Step 3: Run tests
mvn test
# Step 4: Package (create JAR)
mvn package
# Step 5: Verify JAR
ls target/
```

Q.2) Create a simple project and use Git commands to check status, view log history, see differences, commit, and display chronological history

```
# Step 1: Create folder
mkdir gitdemo
cd gitdemo
echo "print('Git Demo Practical')" > app.py
# Step 2: Initialize and commit
git init
git add app.py
git commit -m "Initial commit Added app.py"
# Step 3: Check status and log
git status
git log --oneline
# Step 4: Modify file
echo "print('Version 2 update')" >> app.py
# Step 5: Show diff
git diff
# Step 6: Commit changes
git add app.py
git commit -m "Updated to version 2"
# Step 7: Display chronological history
git log --oneline
```

Practical No-5 And 6

Q.1) Create a simple project, push it to a remote repository on Bitbucket, and create a new branch. Merge this branch into the main branch and display a chronological history of commits.

```
# Step 1: Create project
mkdir bitbucketproject
cd bitbucketproject
echo "print('Bitbucket Project')" > app.py
```

```
git init
git add app.py
git commit -m "Initial commit Added app.py"
```

```
# Step 2: Connect Bitbucket repo
git remote add origin https://bitbucket.org/<your-username>/bitbucketproject.git
git push -u origin master
```

```
# Step 3: Create and modify branch
git checkout -b featurebranch
echo "print('New feature added')" >> app.py
git add app.py
git commit -m "Feature branch update"
```

```
# Step 4: Merge into main and push
git checkout master
git merge featurebranch
git push origin master
```

```
# Step 5: Display commit history
git log --oneline
```

Q.2) Create a simple project, push it to a remote repository on GitHub, create a new branch, merge it into the main branch, display chronological commit history, and pull the changes on your local machine.

```
# Step 1: Create project
mkdir githubproject
cd githubproject
echo "print('GitHub Project')" > app.py
```

```
git init
git add app.py
git commit -m "Initial commit Added app.py"
```

```
# Step 2: Connect GitHub repo
git remote add origin https://github.com/<your-username>/githubproject.git
git push -u origin master
```

```
# Step 3: Create branch
git checkout -b featurebranch
echo "print('Added new feature')" >> app.py
git add app.py
git commit -m "Updated app.py in featurebranch"
```

```
# Step 4: Merge and push
git checkout master
git merge featurebranch
git push origin master
```

```
# Step 5: View history
git log --oneline
```

```
# Step 6: Pull changes to local
git pull origin master
```


Practical No-7

Q.1) GitLab: Create project, branch, merge, show history, and pull changes

Step 1: Create a new project folder and file

```
mkdir gitlabproject
```

```
cd gitlabproject
```

```
echo "print('Welcome to GitLab Project')" > app.py
```

Step 2: Initialize Git repository

```
git init
```

Step 3: Add file and commit

```
git add app.py
```

```
git commit -m "Initial commit Added app.py"
```

Step 4: Create repository on GitLab

Login to GitLab → New Project → Name it gitlabproject → Create

Step 5: Connect local repository with GitLab remote

```
git remote add origin https://gitlab.com/<your-username>/gitlabproject.git
```

Step 6: Push code to GitLab

```
git push -u origin master
```

Step 7: Create a new branch

```
git checkout -b featurebranch
```

Step 8: Modify app.py and commit

```
echo "print('Feature branch update')" >> app.py
```

```
git add app.py
```

```
git commit -m "Updated app.py in featurebranch"
```

Step 9: Merge branch into main

```
git checkout master
```

```
git merge featurebranch
```

Step 10: Push merged changes to GitLab

```
git push origin master
```

Step 11: Display chronological commit history

```
git log --oneline
```

Step 12: Pull changes on local machine

```
git pull origin master
```

Result:

A simple project was created, pushed to GitLab, a new branch was merged into main, commit history was displayed, and the latest changes were successfully pulled.

Q.2) Create CI using Webhook and deploy project using Jenkins Execute Shell

Step 1: Install Jenkins (if not already installed)

```
sudo apt update
sudo apt install openjdk-17-jdk -y
wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo apt-key add -
sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ >
/etc/apt/sources.list.d/jenkins.list'
sudo apt update
sudo apt install jenkins -y
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Step 2: Create a simple project

```
mkdir ci-project
cd ci-project
echo "<h1>Hello CI/CD with Jenkins</h1>" > index.html
git init
git add index.html
git commit -m "Initial commit for CI/CD"
git remote add origin https://gitlab.com/<your-username>/ci-project.git
git push -u origin master
```

Step 3: Configure GitLab webhook

- Go to your GitLab repository → Settings → Webhooks
- Paste Jenkins webhook URL: `http://<jenkins-server-ip>:8080/project/<job-name>`
- Select "Push events" → Add webhook

Step 4: Create a new Jenkins job

- Open Jenkins → New Item → Freestyle project → Name: ci-project-build
- Under Source Code Management → Git → Repository URL: `https://gitlab.com/<your-username>/ci-project.git`
- Add Build Trigger → "Build when a change is pushed to GitLab"

Step 5: Configure Execute Shell to deploy project

Sample deploy command

```
cp -r * /var/www/html/
```

Step 6: Save and trigger build

- Push changes to GitLab → Jenkins will automatically build and deploy via webhook

Step 7: Verify deployment

- Open browser → <http://localhost> (or server IP)
- You should see "Hello CI/CD with Jenkins"

Result:

A CI pipeline was successfully configured using GitLab webhook, and the project was deployed automatically via Jenkins Execute Shell.

Practical No-8

Q.1) Bitbucket: Create a new file on a separate branch, make changes, and merge into main

Step 1: Create a new project folder and file

```
mkdir bitbucketbranch
```

```
cd bitbucketbranch
```

```
echo "print('Initial version')" > app.py
```

Step 2: Initialize Git repository and commit

```
git init
```

```
git add app.py
```

```
git commit -m "Initial commit Added app.py"
```

Step 3: Create a new repository on Bitbucket

- Login to Bitbucket → Create repository → Name it bitbucketbranch

Step 4: Connect local repo with Bitbucket

```
git remote add origin https://bitbucket.org/<your-username>/bitbucketbranch.git
```

```
git push -u origin master
```

Step 5: Create a new branch locally

```
git checkout -b featurebranch
```

Step 6: Create a new file and commit changes

```
echo "print('New feature file added')" > feature.py
```

```
git add feature.py
```

```
git commit -m "Added feature.py in featurebranch"
```

Step 7: Push branch to Bitbucket

```
git push -u origin featurebranch
```

Step 8: Merge branch using Bitbucket interface

- Go to Bitbucket → Repository → Branches → Click on featurebranch → Create pull request → Merge into master/main

Step 9: Verify merge locally

```
git checkout master
```

```
git pull origin master
```

```
ls
```

You will see feature.py in your master branch.

Result:

A new file was created on a separate branch, changes were made, and successfully merged into the main branch using the Bitbucket interface.

Q.2) CI/CD pipeline for web application using Jenkins, Git, and local HTTP server

Step 1: Install Jenkins and a web server

```
sudo apt update
sudo apt install openjdk-17-jdk nginx -y
wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo apt-key add -
sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ >
/etc/apt/sources.list.d/jenkins.list'
sudo apt update
sudo apt install jenkins -y
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Step 2: Create a simple web project

```
mkdir ci-web-project
cd ci-web-project
echo "<h1>Hello CI/CD with Jenkins</h1>" > index.html
git init
git add index.html
git commit -m "Initial commit for CI/CD project"
git remote add origin https://github.com/<your-username>/ci-web-project.git
git push -u origin master
```

Step 3: Configure webhook in Git repository

- Go to GitHub or GitLab repo → Settings → Webhooks
- Add Jenkins webhook URL: `http://<jenkins-server-ip>:8080/project/<job-name>`
- Enable "Push events"

Step 4: Create a Jenkins job

- Open Jenkins → New Item → Freestyle project → Name: ci-web-build
- Source Code Management → Git → Repository URL:
`https://github.com/<your-username>/ci-web-project.git`
- Build Triggers → Build when a change is pushed to Git repository (Git webhook)

Step 5: Configure Execute Shell for deployment

Copy project files to Nginx web directory

```
cp -r * /var/www/html/
```

Step 6: Save job and test

- Push changes to GitHub/GitLab → Jenkins will automatically build and deploy
- Open browser → <http://localhost>
- Web page "Hello CI/CD with Jenkins" will appear

Result:

A CI/CD pipeline was successfully set up using Jenkins and Git. Webhook triggers automated builds, and deployment was done to the local HTTP server.

Practical No-9

Q.1) Implement Bitbucket Operations Using Git

Task 1: Create a new repository on Bitbucket and clone it locally

1. Login to Bitbucket → Create new repository → Name it bitbucket-demo → Click Create
2. Clone repository to local machine:

```
git clone https://bitbucket.org/<your-username>/bitbucket-demo.git
cd bitbucket-demo
```

Task 2: Create a file, commit it, and create a branch

```
# Create example.txt
echo "This is a Bitbucket demo file" > example.txt
```

```
# Add and commit
git add example.txt
git commit -m "Initial commit: Added example.txt"
```

```
# Create a new branch
git checkout -b feature
```

Task 3: Push feature branch and create pull request

```
# Push branch to Bitbucket
git push -u origin feature
```

- Go to Bitbucket → Repository → Branches → Click feature → Create Pull Request → Merge into master
- Review and merge the pull request

Task 4: Verify merge locally

```
git checkout master
git pull origin master
ls
```

You should see example.txt in your master branch.

Result:

A new repository was created on Bitbucket, a file was added on a feature branch, the branch was pushed, a pull request was created, reviewed, merged, and verified locally.

Q.2) Set up and manage dependencies in a Maven project

1. Create a Maven project

```
mvn archetype:generate -DgroupId=com.example -DartifactId=maven-demo \
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
cd maven-demo
```

2. Structure of a pom.xml file

A pom.xml contains:

- project → Root element
- modelVersion → POM model version
- groupId → Project group identifier
- artifactId → Project name
- version → Project version
- dependencies → List of libraries the project needs
- build → Build instructions (plugins, packaging, etc.)

3. Add dependencies

Example: JUnit dependency

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

4. Maven build automation

- Compile project

```
mvn compile
```

- Run tests

```
mvn test
```

- Package and generate JAR file

```
mvn package
```

```
ls target/
```

The generated JAR will be in the target folder.

Result:

Maven project was created, dependencies were managed via pom.xml, and build tasks like compiling, testing, and generating JAR files were automated successfully.

Practical No-10

Q.1) Install Docker and create a "Hello, World!" HTML application

Step 1: Install Docker

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
```

Step 2: Create project folder and HTML file

```
mkdir docker-hello
cd docker-hello
echo "<h1>Hello, World! from Docker</h1>" > index.html
```

Step 3: Create a Dockerfile

```
cat <<EOF > Dockerfile
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
EOF
```

Step 4: Build Docker image

```
sudo docker build -t hello-docker .
```

Step 5: Run Docker container and map to localhost:8080

```
sudo docker run -d -p 8080:80 --name hello-container hello-docker
```

Step 6: Verify container is running

```
sudo docker ps
curl http://localhost:8080
```

Step 7: Stop and remove container

```
sudo docker stop hello-container
sudo docker rm hello-container
```

Result:

Docker was installed, an HTML application was containerized using Nginx, and the application was successfully accessed on <http://localhost:8080>.

Q.2) Git and GitHub Repository Management

Task 1: Repository Setup and Initial Commit

Step 1: Create project folder and file

```
mkdir github-project  
cd github-project  
echo "This is a sample project for DevOps practicals" > project.md
```

Step 2: Initialize Git repository and commit

```
git init  
git add project.md  
git commit -m "Initial commit: Added project.md"
```

Step 3: Push to GitHub

```
git remote add origin https://github.com/<your-username>/github-project.git  
git push -u origin master
```

Task 2: Branching and Merging

Step 1: Create a new branch

```
git checkout -b feature-branch
```

Step 2: Make changes in project.md

```
echo "Added feature details in feature-branch" >> project.md  
git add project.md  
git commit -m "Updated project.md in feature-branch"
```

Step 3: Switch back to main and merge

```
git checkout master  
git merge feature-branch
```

Step 4: Push merged main branch to GitHub

```
git push origin master
```

Step 5: Verify commit history

```
git log --oneline
```

Example output:

```
e5c2a8b Updated project.md in feature-branch  
a9b3d6e Initial commit Added project.md
```


Practical No-11

Q.1) Containerize a basic application and deploy it using Docker

Step 1: Install Docker

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
```

Step 2: Create project folder and HTML file

```
mkdir docker-app
cd docker-app
echo "<h1>Hello Docker Application</h1>" > index.html
```

Step 3: Create a Dockerfile

```
cat <<EOF > Dockerfile
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
EOF
```

Step 4: Build Docker image

```
sudo docker build -t docker-app .
```

Step 5: Run Docker container and map port 8080

```
sudo docker run -d -p 8080:80 --name docker-container docker-app
```

Step 6: Verify deployment

```
sudo docker ps
curl http://localhost:8080
```

Step 7: Stop and remove container

```
sudo docker stop docker-container
sudo docker rm docker-container
```

Result:

The basic application was containerized and successfully deployed on Docker. Accessible at <http://localhost:8080>.

Q.2) Git and GitHub Repository Management

Task 1: Repository Setup and Initial Commit

Step 1: Create project folder and file

```
mkdir github-project  
cd github-project  
echo "This is a sample project for DevOps practicals" > project.md
```

Step 2: Initialize Git repository and commit

```
git init  
git add project.md  
git commit -m "Initial commit: Added project.md"
```

Step 3: Push to GitHub

```
git remote add origin https://github.com/<your-username>/github-project.git  
git push -u origin master
```

Task 2: Branching and Merging

Step 1: Create a new branch

```
git checkout -b feature-branch
```

Step 2: Modify project.md

```
echo "Added feature updates in feature-branch" >> project.md  
git add project.md  
git commit -m "Updated project.md in feature-branch"
```

Step 3: Merge branch into main

```
git checkout master  
git merge feature-branch
```

Step 4: Push updated main branch to GitHub

```
git push origin master
```

Step 5: Verify commit history

```
git log --oneline
```

Example output:

```
e5c2a8b Updated project.md in feature-branch  
a9b3d6e Initial commit Added project.md
```

Practical No-12

Q.1) Applying CI Principles to Web Development Using Jenkins and Git

Step 1: Install Jenkins and Git

```
sudo apt update
sudo apt install openjdk-17-jdk git -y
wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo apt-key add -
sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ >
/etc/apt/sources.list.d/jenkins.list'
sudo apt update
sudo apt install jenkins -y
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Step 2: Create a simple web project

```
mkdir ci-web-project
cd ci-web-project
echo "<h1>Hello CI/CD with Jenkins</h1>" > index.html
git init
git add index.html
git commit -m "Initial commit for CI/CD project"
git remote add origin https://github.com/<your-username>/ci-web-project.git
git push -u origin master
```

Step 3: Configure Jenkins job

1. Open Jenkins: <http://localhost:8080>
2. New Item → Freestyle project → Name: ci-web-build
3. Source Code Management → Git → Repository URL:
<https://github.com/<your-username>/ci-web-project.git>
4. Build Triggers → "Build when a change is pushed to Git repository"
5. Build → Execute shell → Deploy project

```
cp -r * /var/www/html/
```

Step 4: Configure GitHub webhook

- Go to GitHub repo → Settings → Webhooks
- Payload URL: <http://<jenkins-server-ip>:8080/github-webhook/>
- Select "Push events" → Add webhook

Step 5: Test CI

- Push a change to GitHub → Jenkins automatically builds and deploys
- Open browser → <http://localhost> → Verify HTML page

Result: CI pipeline was configured using Jenkins and Git. Webhook automatically triggers build and deployment.

Q.2) Containerize a basic application and deploy it using Docker

Step 1: Install Docker

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
```

Step 2: Create project folder and HTML file

```
mkdir docker-app
cd docker-app
echo "<h1>Hello Docker Application</h1>" > index.html
```

Step 3: Create Dockerfile

```
cat <<EOF > Dockerfile
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
EOF
```

Step 4: Build Docker image

```
sudo docker build -t docker-app .
```

Step 5: Run Docker container and map port 8080

```
sudo docker run -d -p 8080:80 --name docker-container docker-app
```

Step 6: Verify deployment

```
sudo docker ps
curl http://localhost:8080
```

Step 7: Stop and remove container

```
sudo docker stop docker-container
sudo docker rm docker-container
```

Result: The basic application was containerized and successfully deployed via Docker.
Accessible at <http://localhost:8080>.

Practical No-13

Q.1) Bitbucket Repository and Branch Management

Task 1: Repository Setup and Branching

Step 1: Create a repository on Bitbucket

- Login to Bitbucket → Create repository → Name: bitbucket-dev

Step 2: Clone repository locally

```
git clone https://bitbucket.org/<your-username>/bitbucket-dev.git
cd bitbucket-dev
```

Step 3: Create a new branch and add a file

```
git checkout -b development
echo "print('This is the development branch')" > dev.py
git add dev.py
git commit -m "Added dev.py in development branch"
```

Step 4: Push development branch to Bitbucket

```
git push -u origin development
```

Task 2: Pull Request and Code Review

1. Go to Bitbucket → Repository → Branches → Click development → Create Pull Request
2. Select main as target branch → Assign a reviewer → Add comments → Merge pull request

Step 5: Verify merge locally

```
git checkout master
git pull origin master
ls
```

You will see dev.py merged into the main branch.

Result: A repository was created on Bitbucket, a development branch was pushed, and a pull request was created and merged.

Q.2) Automated Deployment with Jenkins and GitHub

Task 1: Configuring Jenkins with GitHub

Step 1: Install Jenkins

```
sudo apt update
sudo apt install openjdk-17-jdk git -y
```

```
wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo apt-key add -
sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ >
/etc/apt/sources.list.d/jenkins.list'
sudo apt update
sudo apt install jenkins -y
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Step 2: Integrate Jenkins with GitHub

- Open Jenkins: <http://localhost:8080>
- Create new Freestyle project → Source Code Management → Git → Repository URL: <https://github.com/<your-username>/ci-web-project.git>
- Build Triggers → "Build when a change is pushed to GitHub"

Task 2: Creating a CI/CD Pipeline

Step 1: Create Jenkins Pipeline

- Jenkins → New Item → Pipeline → Name: ci-cd-web

Step 2: Pipeline Script (Example)

```
pipeline {
  agent any
  stages {
    stage('Clone') {
      steps {
        git 'https://github.com/<your-username>/ci-web-project.git'
      }
    }
    stage('Build') {
      steps {
        echo "Building web application..."
        # For HTML, this is minimal; for Java or Node, you can add compile/build commands
      }
    }
    stage('Archive') {
      steps {
        archiveArtifacts artifacts: '**/*', fingerprint: true
      }
    }
    stage('Deploy') {
      steps {
        sh 'cp -r * /var/www/html/'
      }
    }
  }
}
```

```

    }
  }
  post {
    success {
      echo "Build successful!"
    }
    failure {
      echo "Build failed!"
    }
  }
}
}

```

Step 3: Save Pipeline

- This will clone the repo, build the app, archive artifacts, and deploy to /var/www/html/.
- Task 3: Adding Deployment Step & GitHub Webhook

Step 1: Add Deployment (already in pipeline above)

```
sh 'cp -r * /var/www/html/'
```

Step 2: Configure GitHub Webhook

- GitHub repo → Settings → Webhooks → Payload URL:
http://<jenkins-server-ip>:8080/github-webhook/
- Content type: application/json
- Events: Push → Add webhook

Step 3: Test CI/CD

- Push a change to GitHub → Jenkins automatically builds, archives artifacts, and deploys
- Open browser → <http://localhost> → Verify web application

Result:

Automated CI/CD pipeline was configured with Jenkins, build triggered via GitHub webhook, and deployment to a local HTTP server completed successfully.

Practical No-14

Q.1) Bitbucket Repository Management

Task 1: Creating and Configuring a Bitbucket Repository

Step 1: Create a new repository on Bitbucket

- Login to Bitbucket → New Repository → Name: bitbucket-practical → Add description → Create

Step 2: Clone repository locally

```
git clone https://bitbucket.org/<your-username>/bitbucket-practical.git
cd bitbucket-practical
```

Step 3: Add README.md and push

```
echo "# Bitbucket Practical Repository" > README.md
git add README.md
git commit -m "Added README.md"
git push -u origin master
```

Task 2: Setting Up Access Controls

Step 1: Invite collaborator

- Bitbucket → Repository Settings → User and Group Access → Invite user → Set access to Read

Step 2: Update collaborator access

- Change access to Write → Collaborator can now push to the repository

Step 3: Document access changes

```
echo "Collaborator access updated from Read to Write" > access_control.txt
git add access_control.txt
git commit -m "Documented access control changes"
git push origin master
```

Task 3: Working with Branches

Step 1: Create a new branch

```
git checkout -b feature/update-readme
```


Step 2: Update README.md

```
echo "Updated README for feature branch" >> README.md
git add README.md
git commit -m "Updated README in feature branch"
git push -u origin feature/update-readme
```

Step 3: Create Pull Request and merge

- Go to Bitbucket → Repository → Branches → Click feature/update-readme → Create PR → Merge into main

Step 4: Verify merge locally

```
git checkout master
git pull origin master
cat README.md
```

Result: A Bitbucket repository was created, access controls were configured, a feature branch was added, changes were merged, and everything verified locally.

Q.2) Applying CI/CD Principles to Web Development Using Jenkins, Git, and Local HTTP Server

Step 1: Install Jenkins and web server

```
sudo apt update
sudo apt install openjdk-17-jdk git nginx -y
wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo apt-key add -
sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ >
/etc/apt/sources.list.d/jenkins.list'
sudo apt update
sudo apt install jenkins -y
sudo systemctl start jenkins
sudo systemctl enable jenkins
sudo systemctl start nginx
```

Step 2: Create a simple web project

```
mkdir ci-web-project
cd ci-web-project
echo "<h1>Hello CI/CD with Jenkins</h1>" > index.html
git init
git add index.html
git commit -m "Initial commit for CI/CD project"
git remote add origin https://github.com/<your-username>/ci-web-project.git
git push -u origin master
```

Step 3: Configure Jenkins Job

1. Open Jenkins: <http://localhost:8080>
2. New Item → Freestyle Project → Name: ci-web-build
3. Source Code Management → Git → Repository URL:
<https://github.com/<your-username>/ci-web-project.git>
4. Build Triggers → "Build when a change is pushed to Git repository"
5. Build → Execute Shell → Deploy project:

```
cp -r * /var/www/html/
```

Step 4: Configure GitHub Webhook

- GitHub → Settings → Webhooks → Payload URL:
<http://<jenkins-server-ip>:8080/github-webhook/>
- Content type: application/json
- Events: Push → Add webhook

Step 5: Test CI/CD

- Push changes to GitHub → Jenkins automatically builds and deploys
- Open browser → <http://localhost> → Verify HTML page

Result:

CI/CD pipeline was configured using Jenkins, Git, and a local HTTP server. Build and deployment were automated via webhook.

Practical No-15

Q.1) Bitbucket Repository and Branch Management

Task 1: Repository Setup and Branching

Step 1: Create repository on Bitbucket

- Login to Bitbucket → New Repository → Name: bitbucket-dev → Create

Step 2: Clone repository locally

```
git clone https://bitbucket.org/<your-username>/bitbucket-dev.git
cd bitbucket-dev
```

Step 3: Create a branch, add file, and commit

```
git checkout -b development
echo "print('Development branch file')" > dev.py
git add dev.py
git commit -m "Added dev.py in development branch"
```

Step 4: Push development branch to Bitbucket

```
git push -u origin development
```

Task 2: Pull Request and Code Review

1. Go to Bitbucket → Repository → Branches → Click development → Create Pull Request
2. Select main as the target branch → Assign reviewer or self-review → Add comments → Merge

Step 5: Verify merge locally

```
git checkout master
git pull origin master
ls
dev.py should appear in the main branch.
```

Result: A repository was created, a development branch was pushed, and a pull request was successfully merged.

Q.2) Create a simple project and perform Git operations

Step 1: Create project folder and file

```
mkdir git-project  
cd git-project  
echo "print('Hello Git Project')" > app.py
```

Step 2: Initialize Git repository

```
git init
```

Step 3: Stage files and commit

```
git add app.py  
git commit -m "Initial commit: Added app.py"
```

Step 4: (Optional) View Git log

```
git log --oneline
```

Example output:

```
a1b2c3d Initial commit: Added app.py
```

Result:

A simple project was created, staged, committed, and is ready to push to any remote repository (GitHub, GitLab, or Bitbucket).

Practical No-16

Q.1) Create a simple Java project using Maven. adding dependencies, and Configuring the project's POM file and compile code using maven tool.

1. Create Maven project

```
mvn archetype:generate -DgroupId=com.demo -DartifactId=MyApp  
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

2. Go inside project

```
cd MyApp
```

3. Add dependencies in **pom.xml**

Open **pom.xml** and inside **<dependencies>** tag add this (example: JUnit for testing):

```
<dependencies>  
  <!-- JUnit dependency -->  
  <dependency>  
    <groupId>junit</groupId>  
    <artifactId>junit</artifactId>  
    <version>4.13.2</version>  
    <scope>test</scope>  
  </dependency>  
</dependencies>  
  
<project xmlns="http://maven.apache.org/POM/4.0.0"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
    http://maven.apache.org/xsd/maven-4.0.0.xsd">  
  <modelVersion>4.0.0</modelVersion>  
  
  <groupId>com.demo</groupId>  
  <artifactId>MyApp</artifactId>  
  <version>1.0-SNAPSHOT</version>  
  
  <dependencies>  
    <!-- JUnit dependency -->
```

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
</dependencies>
</project>
```

4. Create Java file

Inside folder src/main/java/com/demo/
create App.java

```
package com.demo;

public class App {
    public static void main(String[] args) {
        System.out.println("Hello Maven Project!");
    }
}
```

5. Compile code using Maven

```
mvn compile
```

6. Run the program

```
mvn exec:java -Dexec.mainClass="com.demo.App"
```

O/P

Hello Maven Project!

Q.2) Applying CI/CD Principles to Web Development Using Jenkins, Git and Local HTTP Server (e.g., Apache or Nginx).

```
# Pull latest code
git clone https://github.com/youruser/webapp.git
cd webapp
```

```
# Optionally build / test (for static nothing, skip)
```

```
# Deploy: copy files to web server folder
sudo cp -r * /var/www/html/
```

You can put a Jenkinsfile in your repo:

```
pipeline {
    agent any

    stages {
        stage('Clone') {
            steps {
                git branch: 'main', url: 'https://github.com/youruser/webapp.git'
            }
        }
        stage('Deploy') {
            steps {
                sh '''
                    # copy to web server folder
                    sudo cp -r * /var/www/html/
                '''
            }
        }
    }
}
```

Git Commands in Linux for This Workflow

These are the Git commands you will frequently use in Linux when doing CI/CD with web development:

```
# Clone the repo (on Jenkins or your local)
git clone https://github.com/youruser/webapp.git
```

```
# If already have repo and want to fetch new changes
cd webapp
git fetch
```

git pull

To commit your changes locally

git add .

git commit -m "Your commit message"

Push changes to remote

git push origin main

If branch workflow:

git checkout -b dev

make changes...

git add .

git commit -m "changes in dev"

git push origin dev

Switch back to main

git checkout main

Merge dev into main locally (if needed)

git merge dev

git push origin main

To view commit history

git log --oneline --graph --decorate --all

In Jenkins shell steps (or pipeline sh), you can use git clone, git pull, etc., exactly as above.

Also useful (in Jenkins or shell) to see changed files:

git diff --name-only HEAD~1

Practical No-17

Q.1) Bitbucket Repository and Branch Management

Task 1: Repository Setup and Branching

- Create a repository on Bitbucket and clone it locally.
 - Create a branch development, add a new file, commit the changes, and push it to the development branch on Bitbucket.
- ### Task 2: Pull Request and Code Review
- In Bitbucket, create a pull request to merge development into main.
 - Assign a reviewer (or self-review) and comment on any changes before merging.

1) Bitbucket Repository and Branch Management

Task 1: Repository Setup and Branching

1. Create a repository on Bitbucket

Go to bitbucket.org

→ Create new repository → Name it my-repo.

Copy the HTTPS URL (example: `https://username@bitbucket.org/username/my-repo.git`)

2. Clone repository locally

```
git clone https://username@bitbucket.org/username/my-repo.git
```

```
cd my-repo
```

3. Create a new branch development

```
git checkout -b development
```

4. Add a new file

```
echo "This is development branch file" > dev.txt
```

```
git add dev.txt
```

```
git commit -m "Added dev.txt in development branch"
```

5. Push to Bitbucket (development branch)

```
git push origin development
```

Task 2: Pull Request and Code Review

This part is done on Bitbucket website.

Go to your repository on Bitbucket.

You'll see a message like: "You've recently pushed a branch: development. Create pull request?" → Click it.

Select: Source branch = development, Target branch = main.

Add title: Merge development into main.

Assign a reviewer (if alone → select yourself).

Add a comment like "Reviewed changes, looks good".

Click Merge.

Q.2) Create a simple project and push on a remote server (like github) using git. and perform some operation. And displays a chronological history of commits.

1. Configure Git (first time only)

```
git config --global user.name "YourName"  
git config --global user.email "your@email.com"
```

2. Create a project

```
mkdir myproject  
cd myproject  
echo "Hello Git Project" > file1.txt  
git init  
git add .  
git commit -m "First commit - added file1"
```

3. Connect to GitHub / Bitbucket remote

```
git remote add origin https://github.com/username/myproject.git  
git branch -M main  
git push -u origin main
```

(replace URL with your repo link)

4. Do some operations

Add new file

```
echo "Second file" > file2.txt  
git add file2.txt  
git commit -m "Added file2"  
git push
```

Update file

```
echo "Update content" >> file1.txt  
git add file1.txt  
git commit -m "Updated file1"  
git push
```

Delete file

```
git rm file2.txt  
git commit -m "Removed file2"  
git push
```

5. Show commit history

```
git log --oneline --graph --decorate --all
```

Practical No-18

Q.1) Applying CI Principles to Web Development Using Jenkins, Git

Sample Jenkinsfile (Declarative Pipeline)

Put this file named Jenkinsfile in your web project repository:

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/youruser/webapp.git'
      }
    }
    stage('Build / Test') {
      steps {
        // If web app has build steps (e.g. with Node, Maven, etc.)
        // Otherwise you can just echo
        sh 'echo Building the web project'
        // Example: npm install, npm run test etc.
      }
    }
    stage('Deploy') {
      steps {
        // Copy files to web server directory
        sh 'sudo cp -r * /var/www/html/'
      }
    }
  }
}
```

Git Commands for this CI Workflow

Here are the Git commands you will run in Linux terminal when using this CI setup:

```
# Clone the web project repo
git clone https://github.com/youruser/webapp.git
cd web app
```

```
# Create a new branch (feature or dev)
```

```
git checkout -b feature1

# Make changes (edit or add files)
git add .
git commit -m "Implement feature1 changes"

# Push branch to remote
git push origin feature1

# Merge changes into main locally (optional)
git checkout main
git merge feature1
git push origin main

# Or directly push commits to main if working there
# git add .
# git commit -m "Update web files"
# git push origin main

# To view commit history
git log --oneline --graph --decorate --all
```

Q.2) Git and GitHub Repository Management Objective: To manage repositories, branching, and merging in Git and GitHub.

Task 1: Repository Setup and Initial Commit

- Set up a local Git repository and create a file named project.md with a brief description of a hypothetical project.
- Initialize the repository, add project.md, commit the changes, and push to a GitHub repository.

Task 2: Branching and Merging

- Create a new branch called feature-branch and make additional changes to project.md.
- Commit the changes in the feature-branch, switch back to main, and merge feature-branch into main.
- Push the updated main branch to GitHub, ensuring the merge is reflected.

Task 1: Repository Setup and Initial Commit

```
# Create a folder and go into it
mkdir myproject
```

```
cd myproject
```

Create the file with a brief description

```
echo "This is a hypothetical project to practice Git & GitHub repository management." >  
project.md
```

Initialize Git repo

```
git init
```

Stage the file

```
git add project.md
```

Commit it

```
git commit -m "Initial commit: add project.md with project description"
```

Add remote GitHub repository

```
# (replace <username> and <repo> with actual)
```

```
git remote add origin https://github.com/<username>/<repo>.git
```

If your default branch should be main

```
git branch -M main
```

Push the commit to GitHub

```
git push -u origin main
```

Task 2: Branching and Merging

Create a new branch

```
git checkout -b feature-branch
```

Make additional changes to project.md

```
echo "Add new feature details: this project will include CI/CD, version control, and more." >>  
project.md
```

Stage and commit the change on feature-branch

```
git add project.md
```

```
git commit -m "Add feature details in project.md on feature-branch"
```

Switch back to main

```
git checkout main
```

Merge the feature-branch into main

```
git merge feature-branch
```

Optionally delete feature branch locally

```
git branch -d feature-branch
```

Push updated main branch to GitHub (merge reflected there too)

```
git push origin main
```

Practical No-19

Q.1) Git and GitHub Repository Management Objective: To manage repositories, branching, and merging in Git and GitHub.

Task 1: Repository Setup and Initial Commit

- Set up a local Git repository and create a file named project.md with a brief description of a hypothetical project.
- Initialize the repository, add project.md, commit the changes, and push to a GitHub repository.

Task 2: Branching and Merging

- Create a new branch called feature-branch and make additional changes to project.md.
- Commit the changes in the feature-branch, switch back to main, and merge feature-branch into main.
- Push the updated main branch to GitHub, ensuring the merge is reflected.

Task 1: Repository Setup and Initial Commit

Create a new directory and navigate into it

```
mkdir myproject
```

```
cd myproject
```

Create the project.md file with a brief description

```
echo "This is a hypothetical project to practice Git and GitHub repository management." >  
project.md
```

Initialize a new Git repository

```
git init
```

Stage the project.md file for commit

```
git add project.md
```

Commit the staged file with a message

```
git commit -m "Initial commit: add project.md with project description"
```

Add the remote GitHub repository (replace <username> and <repo> with actual details)

```
git remote add origin https://github.com/<username>/<repo>.git
```

Set the default branch to 'main'

```
git branch -M main
```

Push the commit to the GitHub repository

```
git push -u origin main
```

Task 2: Branching and Merging

Create and switch to a new branch named 'feature-branch'

```
git checkout -b feature-branch
```

Make changes to project.md (e.g., add more details)

```
echo "This project will include CI/CD, version control, and more." >> project.md
```

Stage the changes for commit

```
git add project.md
```

Commit the changes in 'feature-branch'

```
git commit -m "Add feature details in project.md on feature-branch"
```

Switch back to the 'main' branch

```
git checkout main
```

Merge 'feature-branch' into 'main'

```
git merge feature-branch
```

Push the updated 'main' branch to GitHub

```
git push origin main
```

Optionally, delete the 'feature-branch' locally

```
git branch -d feature-branch
```

Q.2) Create a simple project, push it to a remote repository on GitHub, and create a new branch. Merge this branch into the main branch and display a chronological history of commits.

Ans

Task 1: Repository Setup and Initial Commit

Create a new directory and navigate into it

```
mkdir myproject
```

```
cd myproject
```

Create a file named project.md with a brief description

```
echo "This is a hypothetical project to practice Git and GitHub repository management."
```

```
> project.md
```

Initialize a new Git repository

```
git init
```

Stage the project.md file for commit

git add project.md

Commit the staged file with a message

git commit -m "Initial commit: add project.md with project description"

Add the remote GitHub repository (replace <username> and <repo> with actual details)

git remote add origin https://github.com/<username>/<repo>.git

Set the default branch to 'main'

git branch -M main

Push the commit to the GitHub repository

git push -u origin main

Task 2: Branching and Merging

Create and switch to a new branch named 'feature-branch'

git checkout -b feature-branch

Make changes to project.md (e.g., add more details)

echo "This project will include CI/CD, version control, and more." >> project.md

Stage the changes for commit

git add project.md

Commit the changes in 'feature-branch'

git commit -m "Add feature details in project.md on feature-branch"

Push the 'feature-branch' to GitHub

git push -u origin feature-branch

Switch back to the 'main' branch

git checkout main

Merge 'feature-branch' into 'main'

git merge feature-branch

Push the updated 'main' branch to GitHub

git push origin main

Optionally, delete the 'feature-branch' locally

git branch -d feature-branch

Viewing Commit History

git log --oneline --graph --decorate --all

Practical No-20

Q.1) Applying CI Principles to Web Development Using Jenkins, Git

Task: Set Up Continuous Integration (CI) Pipeline

1. Create a Simple Web Project

Create project directory and navigate into it

```
mkdir my-web-app
```

```
cd my-web-app
```

Initialize a Git repository

```
git init
```

Create a simple HTML file

```
echo "<html><body><h1>Hello, Jenkins CI!</h1></body></html>" > index.html
```

Stage and commit the changes

```
git add index.html
```

```
git commit -m "Initial commit: Add index.html"
```

Create a GitHub repository (replace <username> and <repo> with actual details)

```
git remote add origin https://github.com/<username>/<repo>.git
```

Push the local repository to GitHub

```
git push -u origin main
```

2. Set Up Jenkins Pipeline

a. Install Necessary Plugins in Jenkins

Git Plugin: Allows Jenkins to interact with Git repositories.

GitHub Integration Plugin: Enables Jenkins to receive webhook notifications from GitHub.

b. Create a New Pipeline Job

Log in to Jenkins.

Click on New Item.

Enter a name for your job (e.g., my-web-app-pipeline).

Select Pipeline and click OK.

c. Configure the Pipeline

In the Pipeline section, set Definition to Pipeline script from SCM.

Set SCM to Git.

Enter your GitHub repository URL in Repository URL.

Set Branch Specifier to */main.

In Script Path, enter Jenkinsfile.

d. Create a Jenkinsfile in Your Repository

```
pipeline {  
  agent any  
  
  stages {  
    stage('Checkout') {  
      steps {  
        git branch: 'main', url: 'https://github.com/<username>/<repo>.git'  
      }  
    }  
    stage('Deploy') {  
      steps {  
        sh 'cp index.html /var/www/html/'  
      }  
    }  
  }  
}
```

Replace <username> and <repo> with your actual GitHub username and repository name.

3. Set Up GitHub Webhook

Navigate to your GitHub repository.

Go to Settings > Webhooks > Add webhook.

Enter your Jenkins server URL followed by /github-webhook/ (e.g., http://<jenkins-server>/github-webhook/).

Set Content type to application/json.

Choose Just the push event.

Click Add webhook.

4. Push Changes to Trigger Jenkins Build

Make changes to your project (e.g., update index.html)

```
echo "<html><body><h1>Updated: Hello, Jenkins CI!</h1></body></html>" >
index.html
```

```
# Stage and commit the changes
git add index.html
git commit -m "Update index.html"
```

```
# Push the changes to GitHub
git push origin main
```

Q.2) Create a simple project, push it to a remote repository on Github , and create a new branch. Merge this branch into the main branch and display a chronological history of commits.and Pull the changes on your local machine.

Task 1: Repository Setup and Initial Commit

```
# Create a new directory and navigate into it
mkdir myproject
cd myproject
```

```
# Create a file named project.md with a brief description
echo "This is a hypothetical project to practice Git and GitHub repository management."
> project.md
```

```
# Initialize a new Git repository
git init
```

```
# Stage the project.md file for commit
git add project.md
```

```
# Commit the staged file with a message
git commit -m "Initial commit: add project.md with project description"
```

```
# Add the remote GitHub repository (replace <username> and <repo> with actual
details)
git remote add origin https://github.com/<username>/<repo>.git
```

```
# Set the default branch to 'main'
git branch -M main
```

```
# Push the commit to the GitHub repository
git push -u origin main
```

Task 2: Branching and Merging

```
# Create and switch to a new branch named 'feature-branch'
git checkout -b feature-branch
```

```
# Make changes to project.md (e.g., add more details)
echo "This project will include CI/CD, version control, and more." >> project.md
```

```
# Stage the changes for commit
git add project.md
```

```
# Commit the changes in 'feature-branch'
git commit -m "Add feature details in project.md on feature-branch"
```

```
# Push the 'feature-branch' to GitHub
git push -u origin feature-branch
```

```
# Switch back to the 'main' branch
git checkout main
```

```
# Merge 'feature-branch' into 'main'
git merge feature-branch
```

```
# Push the updated 'main' branch to GitHub
git push origin main
```

```
# Optionally, delete the 'feature-branch' locally
git branch -d feature-branch
```

Viewing Commit History

To display a chronological history of commits across all branches:

```
git log --oneline --graph --decorate --all
```

Practical No-21

Q.1) Create a new file on a separate branch, make some changes to this file, and then merge these changes into the main branch using bitBucket interface.

Ans

Step 1: Create a New Branch in Bitbucket

Navigate to your repository in Bitbucket

Click on Branches in the left sidebar.

Click the Create branch button.

In the Create branch dialog:

From branch: Select main (or your default branch).

Branch name: Enter a descriptive name, e.g., feature/add-new-file.

Click Create to create the new branch.

Step 2: Add a New File in the New Branch

Switch to the newly created branch by selecting it from the branch dropdown in the top-left corner.

Navigate to the Source tab.

Click the Create file button.

In the Create file dialog:

File name: Enter newfile.txt.

File content: Add some content, e.g., This is a new file added in the feature branch.

Below the editor, in the Commit changes section:

Commit message: Enter Add newfile.txt with initial content.

Ensure the Commit to branch is set to your new branch (feature/add-new-file).

Click Commit to save the new file to the branch.

Step 3: Create a Pull Request to Merge into Main

Navigate to the Pull requests tab.

Click the Create pull request button.

In the Create pull request dialog:

Source: Select your new branch (feature/add-new-file).

Destination: Select main (or your default branch).

Title: Enter a descriptive title, e.g., Merge newfile.txt into main.

Description: Provide details about the changes, e.g., This pull request adds a new file, newfile.txt, to the repository.

Click Create pull request to open the PR for review.

Step 4: Merge the Pull Request

In the pull request view, review the changes and ensure everything is correct.

Click the Merge button.

In the merge dialog:

Merge strategy: Choose the appropriate strategy (e.g., Merge commit).

Delete source branch: Optionally, check this box to delete the source branch after merging.

Click Merge to complete the process.

Step 5: Pull the Changes to Your Local Machine

Open your terminal.

Navigate to your local repository directory.

Ensure you're on the main branch:

git checkout main

Pull the latest changes from the remote repository:

git pull origin main

Q.2) Applying CI Principles to Web Development Using Jenkins, Git

Step 1: Set Up a Simple Web Project

Create Project Directory and Initialize Git Repository:

```
mkdir my-web-app  
cd my-web-app  
git init
```

Create a Simple HTML File:

```
echo "<html><body><h1>Hello, Jenkins CI!</h1></body></html>" > index.html
```

Stage and Commit the Changes:

```
git add index.html  
git commit -m "Initial commit: Add index.html"
```

Create a Remote Repository on GitHub:

Navigate to GitHub
and create a new repository named my-web-app.

Add Remote and Push Changes:

```
git remote add origin https://github.com/<username>/my-web-app.git
git branch -M main
git push -u origin main
```

Replace <username> with your GitHub username.

Step 2: Set Up Jenkins for CI

Install Jenkins:

On your Linux machine, install Jenkins by following the official installation guide

Start Jenkins:

```
sudo systemctl start jenkins
```

Access Jenkins:

Open your browser and navigate to <http://localhost:8080>.

Follow the setup wizard to unlock Jenkins and install the suggested plugins.

Install Git Plugin:

In Jenkins, go to Manage Jenkins > Manage Plugins.

Under the Available tab, search for Git Plugin and install it.

Step 3: Create a Jenkins Pipeline

Create a New Pipeline Job:

In Jenkins, click on New Item.

Enter a name for your job (e.g., my-web-app-pipeline).

Select Pipeline and click OK.

Configure Pipeline:

Under Pipeline section, set Definition to Pipeline script from SCM.

Set SCM to Git.

Enter your GitHub repository URL in Repository URL.

Set Branch Specifier to */main.

Create a Jenkinsfile:

In your project directory, create a file named Jenkinsfile with the following content:

```

pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/<username>/my-web-app.git'
      }
    }
    stage('Deploy') {
      steps {
        sh 'cp index.html /var/www/html/'
      }
    }
  }
}

```

Replace <username> with your GitHub username.

Commit and Push Jenkinsfile:

```

git add Jenkinsfile
git commit -m "Add Jenkinsfile for CI pipeline"
git push origin main

```

Step 4: Configure GitHub Webhook

Generate Jenkins API Token:

In Jenkins, go to People > your username > Configure.

Under API Token, click Show API Token and copy it.

Set Up Webhook in GitHub:

In your GitHub repository, go to Settings > Webhooks > Add webhook.

In Payload URL, enter `http://<jenkins-server>/github-webhook/`.

Set Content type to `application/json`.

In Secret, enter your Jenkins API token.

Select Just the push event.

Click Add webhook.

Replace <jenkins-server> with your Jenkins server's address.

Step 5: Push Changes to Trigger Jenkins Build

Make Changes to Your Project:

```
echo "<html><body><h1>Updated: Hello, Jenkins CI!</h1></body></html>" >  
index.html
```

Stage and Commit the Changes:

```
git add index.html  
git commit -m "Update index.html"
```

Push Changes to GitHub:

```
git push origin main
```

This push will trigger Jenkins to execute the pipeline, deploying the updated index.html to your web server.

Step 6: Monitor Jenkins Build

In Jenkins, navigate to your pipeline job.

Under Build History, click on the latest build to view its progress and logs.

Practical No-22

Q.1) Create a simple Java project using Maven. adding dependencies, and Configuring the project's POM file and compile code using maven tool.

Step 1: Install Maven

Before starting, ensure that Maven is installed on your Linux machine. Open your terminal and run:

```
mvn --version
```

If Maven is not installed, you can install it using:

```
sudo apt update  
sudo apt install maven
```

Step 2: Create a Simple Java Project

Navigate to your desired directory:

```
cd ~/Projects
```

Generate a Maven project using the archetype:generate goal:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=my-app  
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

This command creates a new Maven project with the following structure:

```
my-app/  
├── pom.xml  
└── src/  
    ├── main/  
    │   └── java/  
    │       ├── com/  
    │       │   └── example/  
    │       │       └── App.java
```

Step 3: Add Dependencies to pom.xml

Navigate to the my-app directory:

```
cd my-app
```

Open the pom.xml file in a text editor:

```
nano pom.xml
```

Add the following dependencies inside the <dependencies> section:

```
<dependencies>
  <!-- JUnit for testing -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.9.1</version>
    <scope>test</scope>
  </dependency>

  <!-- SLF4J for logging -->
  <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>2.0.0-alpha1</version>
  </dependency>
</dependencies>
```

Save and close the editor (Ctrl + X, then Y, and Enter).

Step 4: Compile the Project Using Maven

To compile the project, run:

```
mvn compile
```

This command compiles the source code and places the compiled .class files in the target/classes directory.

Step 5: Package the Project into a JAR File

To package the compiled code into a JAR file, run:

```
mvn package
```

This command creates a JAR file located in the target directory, named my-app-1.0-SNAPSHOT.jar.

Step 6: Run the JAR File

To run the generated JAR file, use:

```
java -cp target/my-app-1.0-SNAPSHOT.jar com.example.App
```

This command executes the App class from the JAR file.

Step 7: Run Tests

To execute the tests defined in your project, run:

```
mvn test
```

This command runs the tests and displays the results in the terminal.

Step 8: Install the JAR to Local Repository

If you want to install the JAR file to your local Maven repository, run:

```
mvn install
```

Q.2) Install Docker on your system and create a simple "Hello, World!" application using HTML. Create a Dockerfile to containerize the application, using an official web server image as the base. Build the Docker image, tag it, and run a container, making the application accessible on a local port (e.g., <http://localhost:8080>).

Step 1: Install Docker on Linux

To install Docker on your Linux system, follow these steps:

Update your package index:

```
sudo apt update
```

Install required packages:

```
sudo apt install apt-transport-https ca-certificates curl software-properties-common
```

Add Docker's official GPG key:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

Set up the stable repository:

```
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Update the package index again:

```
sudo apt update
```

Install Docker Engine:

```
sudo apt install docker-ce docker-ce-cli containerd.io
```

Start and enable Docker:

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

Verify Docker installation:

```
sudo docker --version
```

This should display the installed Docker version.

Step 2: Create a Simple "Hello, World!" HTML Application

Create a project directory:

```
mkdir hello-world-app
```

```
cd hello-world-app
```

Create an index.html file:

```
nano index.html
```

Add the following content:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hello, World!</title>
</head>
<body>
  <h1>Hello, World!</h1>
</body>
</html>
```

Save and exit (Ctrl + X, then Y, and Enter).

Step 3: Create a Dockerfile to Containerize the Application

Create a Dockerfile:

nano Dockerfile

Add the following content:

```
# Use the official Nginx image from the Docker Hub
FROM nginx:alpine
```

```
# Copy the index.html file to the Nginx server's default directory
COPY index.html /usr/share/nginx/html/index.html
```

```
# Expose port 80
EXPOSE 80
```

Save and exit (Ctrl + X, then Y, and Enter).

Step 4: Build and Run the Docker Container

Build the Docker image:

```
sudo docker build -t hello-world-app .
```

This command builds the Docker image and tags it as hello-world-app.

Run the Docker container:

```
sudo docker run -d -p 8080:80 hello-world-app
```

This command runs the container in detached mode (-d), maps port 80 of the container to port 8080 on your local machine (-p 8080:80), and uses the hello-world-app image.

Step 5: Access the Application

Open your web browser and navigate to:

<http://localhost:8080>

You should see the "Hello, World!" message displayed.

Step 6: Manage Docker Containers

List running containers:

```
sudo docker ps
```

Stop the running container:

```
sudo docker stop <container_id>
```

Replace <container_id> with the actual container ID obtained from the docker ps command.

Remove the stopped container:

```
sudo docker rm <container_id>
```

Remove the Docker image:

```
sudo docker rmi hello-world-app
```

Practical No-23

Q.1) Applying CI Principles to Web Development Using Jenkins, Git

Step 1: Install Jenkins on Linux

Update your package index:

```
sudo apt update
```

Install Java Development Kit (JDK):

```
sudo apt install openjdk-11-jdk
```

Add Jenkins repository and key:

```
wget -q -O - https://pkg.jenkins.io/jenkins.io.key | sudo tee  
/etc/apt/trusted.gpg.d/jenkins.asc  
sudo sh -c 'wget -q -O /etc/apt/sources.list.d/jenkins.list  
https://pkg.jenkins.io/debian/jenkins.io.list'
```

Update package index again:

```
sudo apt update
```

Install Jenkins:

```
sudo apt install jenkins
```

Start Jenkins service:

```
sudo systemctl start jenkins  
sudo systemctl enable jenkins
```

Access Jenkins: Open your browser and go to <http://localhost:8080>. Follow the setup wizard to complete the installation.

Step 2: Install Git on Linux

Install Git:

```
sudo apt install git
```

Verify installation:

```
git --version
```


Step 3: Create a Sample Web Application

Create a project directory:

```
mkdir my-web-app  
cd my-web-app
```

Initialize a Git repository:

```
git init
```

Create an index.html file:

```
nano index.html
```

Add the following content:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>My Web App</title>  
</head>  
<body>  
  <h1>Hello, World!</h1>  
</body>  
</html>
```

Save and exit (Ctrl + X, then Y, and Enter).

Commit the changes:

```
git add index.html  
git commit -m "Initial commit"
```

Step 4: Set Up GitHub Repository

Create a new repository on GitHub.

Add the remote origin:

```
git remote add origin https://github.com/your-username/my-web-app.git
```

Push the code to GitHub:

```
git push -u origin master
```

Step 5: Configure Jenkins to Monitor GitHub Repository

Install necessary plugins:

Go to Jenkins Dashboard → Manage Jenkins → Manage Plugins.

Install the following plugins:

Git plugin

GitHub plugin

GitHub Integration plugin

Create a new Jenkins job:

Go to Jenkins Dashboard → New Item.

Enter a name for your job (e.g., MyWebAppBuild).

Select "Freestyle project" and click OK.

Configure Source Code Management:

Under "Source Code Management", select "Git".

Enter your GitHub repository URL: `https://github.com/your-username/my-web-app.git`.

Add credentials if required.

Set up Build Triggers:

Under "Build Triggers", check "GitHub hook trigger for GITScm polling".

Configure Build Steps:

Under "Build", click "Add build step" and select "Execute shell".

Enter the following command to serve the application using Python's HTTP server:

```
python3 -m http.server 8080
```

Save the job.

Step 6: Set Up GitHub Webhook to Trigger Jenkins Build

Generate Jenkins API Token:

Go to Jenkins Dashboard → Your Username → Configure → API Token → Show API Token.

Configure GitHub Webhook:

Go to your GitHub repository → Settings → Webhooks → Add webhook.

Payload URL: `http://your-jenkins-url/github-webhook/`.

Content type: `application/json`.

Secret: Leave blank.

Which events would you like to trigger this webhook? Select "Just the push event".

Click "Add webhook".

Step 7: Test the CI Integration

Make a change to your `index.html` file:

```
nano index.html
```

Modify the content, for example:

<h1>Hello, Jenkins!</h1>

Save and exit.

Commit and push the changes:

```
git add index.html
```

```
git commit -m "Updated greeting"
```

```
git push origin master
```

Verify Jenkins Build:

Go to Jenkins Dashboard → Your Job → Build History.

Ensure a new build is triggered and completes successfully.

Access the Application:

Open your browser and go to <http://localhost:8080>.

You should see the updated greeting message.

Step 8: Clean Up

Stop the Jenkins build process:

In Jenkins, go to your job's workspace and stop the running process.

Remove the job:

Go to Jenkins Dashboard → Your Job → Delete Project.

Q.2) create a simple project And push on remote server (like github) using git. and perform some operation. And displays a chronological history of commits.

Step 1: Create a Simple Project

Create a project directory:

```
mkdir my-web-app
```

```
cd my-web-app
```

Initialize a Git repository:

```
git init
```

Create an index.html file:

```
nano index.html
```

Add the following content:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>My Web App</title>
</head>
<body>
  <h1>Hello, World!</h1>
</body>
</html>
```

Save and exit (Ctrl + X, then Y, and Enter).

Stage and commit the changes:

```
git add index.html
```

```
git commit -m "Initial commit"
```

Step 2: Push to a Remote Repository on GitHub

Create a new repository on GitHub.

Add the remote origin:

```
git remote add origin https://github.com/your-username/my-web-app.git
```

Push the code to GitHub:

```
git push -u origin master
```

This command pushes your local commits to the remote repository on GitHub
GitHub Docs.

Step 3: Perform Some Operations

Create a new branch:

```
git checkout -b feature-branch
```

Make changes to index.html:

```
nano index.html
```

Modify the content, for example:

```
<h1>Welcome to My Web App</h1>
```

Save and exit.

Stage and commit the changes:

```
git add index.html
```

```
git commit -m "Updated greeting message"
```

Push the new branch to GitHub:

```
git push -u origin feature-branch
```

Step 4: Merge the Feature Branch into Main

Switch to the main branch:

```
git checkout main
```

Merge the feature branch:

```
git merge feature-branch
```

Push the updated main branch to GitHub:

```
git push origin main
```

Step 5: Display a Chronological History of Commits

View the commit history:

```
git log --oneline --graph
```

Practical No-24

Q.1) Create CI using Webhook and deploy a project using Jenkins Execute shell.

Ans

Step 1: Create a Simple Project

Initialize a Git repository:

```
mkdir my-web-app  
cd my-web-app  
git init
```

Create an index.html file:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>My Web App</title>  
</head>  
<body>  
  <h1>Hello, World!</h1>  
</body>  
</html>
```

Commit the changes:

```
git add index.html  
git commit -m "Initial commit"
```

Push to GitHub:

Create a new repository on GitHub.

Add the remote origin:

```
git remote add origin https://github.com/your-username/my-web-app.git
```

Push the code:

```
git push -u origin master
```

Step 2: Set Up Jenkins Job

Install Jenkins: Follow the official Jenkins installation guide for your operating system.

Install Required Plugins:

Go to Manage Jenkins > Manage Plugins.

Install the following plugins:

Git plugin

GitHub plugin

GitHub Integration plugin

Create a New Jenkins Job:

Go to New Item.

Enter a name (e.g., MyWebAppBuild).

Select Freestyle project and click OK.

Configure Source Code Management:

Under Source Code Management, select Git.

Enter your GitHub repository URL: <https://github.com/your-username/my-web-app.git>.

Add credentials if required.

Set Build Triggers:

Under Build Triggers, check GitHub hook trigger for GITScm polling.

Define Build Steps:

Under Build, click Add build step and select Execute shell.

Enter the following command to serve the application using Python's HTTP server:

```
python3 -m http.server 8080
```

Save the Job.

Step 3: Configure GitHub Webhook

Generate Jenkins API Token:

Go to Jenkins Dashboard > Your Username > Configure > API Token > Show API Token.

Configure GitHub Webhook:

Go to your GitHub repository > Settings > Webhooks > Add webhook.

Payload URL: <http://your-jenkins-url/github-webhook/>.

Content type: application/json.

Secret: Leave blank.

Which events would you like to trigger this webhook? Select Just the push event.

Click Add webhook.

Step 4: Test the CI Integration

Make a Change:

Edit the index.html file:

```
<h1>Welcome to My Web App</h1>
```

Commit and Push the Changes:

```
git add index.html
```

```
git commit -m "Updated greeting"
```

```
git push origin master
```

Verify Jenkins Build:

Go to Jenkins Dashboard > Your Job > Build History.

Ensure a new build is triggered and completes successfully.

Access the Application:

Open your browser and go to <http://localhost:8080>.

You should see the updated greeting message.

Step 5: Clean Up

Stop the Jenkins Build Process:

In Jenkins, go to your job's workspace and stop the running process.

Remove the Job:

Go to Jenkins Dashboard > Your Job > Delete Project.

Q.2) Applying CI/CD Principles to Web Development Using Jenkins, Git and Local HTTP Server (e.g., Apache or Nginx).

Step 1: Create a Simple Web Application

Initialize a Git Repository:

```
mkdir my-web-app  
cd my-web-app  
git init
```

Create an index.html File:

```
<!DOCTYPE html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>My Web App</title>  
  </head>  
  <body>  
    <h1>Hello, Jenkins!</h1>  
  </body>  
</html>
```

Commit the Changes:

```
git add index.html  
git commit -m "Initial commit"
```

Push to GitHub:

Create a new repository on GitHub.

Add the remote origin:

```
git remote add origin https://github.com/your-username/my-web-app.git
```

Push the code:

```
git push -u origin master
```

Step 2: Set Up Jenkins for CI/CD

Install Jenkins:

Follow the official Jenkins installation guide
for your operating system.

Install Required Plugins:

Go to Manage Jenkins > Manage Plugins.

Install the following plugins:

Git plugin

GitHub plugin

Pipeline plugin

Create a New Jenkins Pipeline Job:

Go to New Item.

Enter a name (e.g., MyWebAppBuild).

Select Pipeline and click OK.

Configure the Pipeline:

Under Pipeline, set the Definition to Pipeline script from SCM.

Set the SCM to Git.

Enter your GitHub repository URL: <https://github.com/your-username/my-web-app.git>.

Add credentials if required.

Set the Branch Specifier to */master.

Create a Jenkinsfile in your repository with the following content:

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/your-username/my-web-app.git'
      }
    }
    stage('Deploy') {
      steps {
        script {
          sh '''
            sudo cp index.html /var/www/html/index.html
            sudo systemctl restart apache2
          '''
        }
      }
    }
  }
}
```

This pipeline script checks out the code from GitHub and deploys it to the Apache server.

Step 3: Configure GitHub Webhook

Generate Jenkins API Token:

Go to Jenkins Dashboard > Your Username > Configure > API Token > Show API Token.

Configure GitHub Webhook:

Go to your GitHub repository > Settings > Webhooks > Add webhook.

Payload URL: `http://your-jenkins-url/github-webhook/`.

Content type: `application/json`.

Secret: Leave blank.

Which events would you like to trigger this webhook? Select Just the push event.

Click Add webhook.

Step 4: Set Up Apache Web Server

Install Apache:

```
sudo apt update
```

```
sudo apt install apache2
```

Configure Apache:

Ensure that Apache is set to serve files from `/var/www/html`.

Set proper permissions for the directory:

```
sudo chown -R www-data:www-data /var/www/html
```

Restart Apache:

```
sudo systemctl restart apache2
```

Step 5: Test the CI/CD Pipeline

Make a Change:

Edit the `index.html` file:

```
<h1>Welcome to My Web App</h1>
```

Commit and Push the Changes:

```
git add index.html
```

```
git commit -m "Updated greeting"
```

```
git push origin master
```

Verify Jenkins Build:

Go to Jenkins Dashboard > Your Job > Build History.

Ensure a new build is triggered and completes successfully.

Access the Application:

Open your browser and go to <http://localhost>.

You should see the updated greeting message.

Step 6: Clean Up

Stop the Jenkins Build Process:

In Jenkins, go to your job's workspace and stop the running process.

Remove the Job:

Go to Jenkins Dashboard > Your Job > Delete Project.

Practical No-25

Q.1) Create a simple project, push it to a remote repository on GitLab, and create a new branch. Merge this branch into the main branch and display a chronological history of commits.

Step 1: Create a Simple Project

Initialize a Git repository:

```
mkdir my-web-app  
cd my-web-app  
git init
```

Create an index.html file:

```
<!DOCTYPE html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>My Web App</title>  
  </head>  
  <body>  
    <h1>Hello, GitLab!</h1>  
  </body>  
</html>
```

Commit the changes:

```
git add index.html  
git commit -m "Initial commit"
```

Step 2: Push to a Remote GitLab Repository

Create a new repository on GitLab:

Go to GitLab.

Click on the New project button.

Choose Create blank project.

Enter a project name (e.g., my-web-app).

Set the visibility (Private, Internal, or Public).

Click Create project.

Add the remote origin:

```
git remote add origin https://gitlab.com/your-username/my-web-app.git
```

Push the code to GitLab:

```
git push -u origin master
```

Step 3: Create a New Branch

Create and switch to a new branch:

```
git checkout -b feature-branch
```

Make changes to index.html:

```
<h1>Welcome to My Web App</h1>
```

Commit the changes:

```
git add index.html
```

```
git commit -m "Updated greeting"
```

Push the new branch to GitLab:

```
git push -u origin feature-branch
```

Step 4: Merge the Feature Branch into Main

Open a merge request on GitLab:

Go to your project on GitLab.

Navigate to Merge Requests > New merge request.

Select feature-branch as the source branch and master (or main) as the target branch.

Click Compare branches and continue.

Add a title and description for the merge request.

Click Submit merge request.

Merge the request:

After reviewing the changes, click Merge to merge the feature branch into the main branch.

Step 5: Display a Chronological History of Commits

View commit history on GitLab:

Go to your project on GitLab.

Navigate to Repository > Commits.

Here, you'll see a chronological list of all commits made to the repository.

View commit history using Git:

```
git log --oneline --graph
```

Q.2) Create CI using Webhook and deploy a project using Jenkins Execute shell.

Ans

Step 1: Create a Simple Project and Push to GitHub

Initialize a Git repository:

```
mkdir my-web-app
```

```
cd my-web-app
```

```
git init
```

Create an index.html file:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
  <head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>My Web App</title>
```

```
  </head>
```

```
  <body>
```

```
    <h1>Hello, GitHub!</h1>
```

```
  </body>
```

```
</html>
```

Commit the changes:

```
git add index.html
```

```
git commit -m "Initial commit"
```

Push to GitHub:

Create a new repository on GitHub.

Add the remote origin:

```
git remote add origin https://github.com/your-username/my-web-app.git
```

Push the code:

```
git push -u origin master
```

Step 2: Set Up Jenkins for CI/CD

Install Jenkins:

Follow the official Jenkins installation guide for your operating system.

Install Required Plugins:

Go to Manage Jenkins > Manage Plugins.

Install the following plugins:

Git plugin

GitHub plugin

GitHub Integration plugin

Create a New Jenkins Job:

Go to New Item.

Enter a name (e.g., MyWebAppBuild).

Select Freestyle project and click OK.

Configure Source Code Management:

Under Source Code Management, select Git.

Enter your GitHub repository URL: <https://github.com/your-username/my-web-app.git>.

Add credentials if required.

Set Build Triggers:

Under Build Triggers, check GitHub hook trigger for GITScm polling.

Define Build Steps:

Under Build, click Add build step and select Execute shell.

Enter the following command to serve the application using Python's HTTP server:

```
python3 -m http.server 8080
```

Save the Job.

Step 3: Configure GitHub Webhook

Generate Jenkins API Token:

Go to Jenkins Dashboard > Your Username > Configure > API Token > Show API Token.

Configure GitHub Webhook:

Go to your GitHub repository > Settings > Webhooks > Add webhook.

Payload URL: `http://your-jenkins-url/github-webhook/`.

Content type: `application/json`.

Secret: Leave blank.

Which events would you like to trigger this webhook? Select Just the push event.

Click Add webhook.

Step 4: Test the CI Integration

Make a Change:

Edit the `index.html` file:

```
<h1>Welcome to My Web App</h1>
```

Commit and Push the Changes:

```
git add index.html
```

```
git commit -m "Updated greeting"
```

```
git push origin master
```

Verify Jenkins Build:

Go to Jenkins Dashboard > Your Job > Build History.

Ensure a new build is triggered and completes successfully.

Access the Application:

Open your browser and go to `http://localhost:8080`.

You should see the updated greeting message.

Step 5: Clean Up

Stop the Jenkins Build Process:

In Jenkins, go to your job's workspace and stop the running process.

Remove the Job:

Go to Jenkins Dashboard > Your Job > Delete Project.